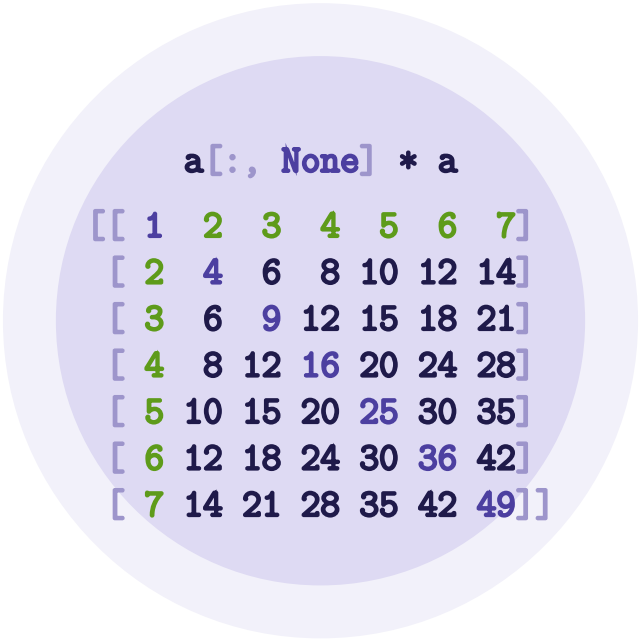
Массивы в numpy

Урок



numpy — библиотека для массивов чисел, и на ней стоит почти всё в курсе: pandas, sklearn, PyTorch. Урок идёт от простого, и каждая новая вещь появляется на маленьком примере. Код — в цветных ячейках, под чертой на белом — ровно то, что он печатает: вывод снят запуском. Ячейки идут подряд, как в ноутбуке, и следующая пользуется тем, что задала предыдущая. Лучше набирать их самому и менять числа: так видно, что от чего зависит.

1 Массив вместо списка

Неделя наблюдений: температура днём, по дням с понедельника. В питоне это список, и чтобы перевести её в градусы Фаренгейта, нужен цикл:

```
t = [3, 5, 4, -1, -2, 0, 6]
f = [x * 1.8 + 32 for x in t]
print(f)
```

```
[37.4, 41.0, 39.2, 30.2, 28.4, 32.0, 42.8]
```

Массив numpy делает то же одной формулой — она сама применяется к каждому элементу:

```
import numpy as np
a = np.array(t)
print(a)
print(a * 1.8 + 32)
```

```
[ 3  5  4 -1 -2  0  6]
[37.4 41.  39.2 30.2 28.4 32.  42.8]
```

`np.array` делает из списка массив. Печатается массив без запятых — по этому его и отличают от списка; `41.` — это `41.0`: нули после точки numpy не пишет.

Ловушка. У списков та же арифметика значит другое: `*` повторяет список, `+` склеивает два. Массив считает поэлементно:

```
print(t * 2)
print(a * 2)
print(t + t)
print(a + a)
```

```
[3, 5, 4, -1, -2, 0, 6, 3, 5, 4, -1, -2, 0, 6]
[ 6 10  8 -2 -4  0 12]
[3, 5, 4, -1, -2, 0, 6, 3, 5, 4, -1, -2, 0, 6]
[ 6 10  8 -2 -4  0 12]
```

Два массива одной длины складываются, вычитаются и делятся тоже поэлементно — по парам, стоящим на одних местах. Ночная температура той же недели:

```
night = np.array([-2, 0, 1, -5, -6, -3, 1])
print(a - night)
print((a + night) / 2)
```

```
[5 5 3 4 4 3 5]
[ 0.5  2.5  2.5 -3.  -4.  -1.5  3.5]
```

Первая строка — перепад между днём и ночью, вторая — средняя за сутки, по каждому дню. Если длины разные, пар не составить, и numpy не гадает, а падает:

```
print(a + np.array([1, 2]))
```

```
ValueError: operands could not be broadcast together
with shapes (7,) (2,)
```

Слово broadcast здесь не случайно: иногда массивы разной длины всё же складываются, и правило для этого будет в разделе о распространении.

Функции numpy тоже бывают поэлементными: `np.abs`, `np.sqrt`, `np.exp`, `np.round`. А бывают такие, что сворачивают массив в одно число:

```
print(np.abs(night))
print(a.sum(), a.min(), a.max())
print(a.mean())
print(len(a))
```

```
[2 0 1 5 6 3 1]
15 -2 6
2.142857142857143
7
```

Пример (неделя одной строкой). Насколько в среднем ночь холоднее дня и сколько градусов между самым тёплым и самым холодным днём:

```
print((a - night).mean())
print(a.max() - a.min())
```

```
4.142857142857143
8
```

Как устроено. Список хранит ссылки на отдельные объекты питона, разбросанные по памяти, и цикл по нему на каждом шаге выясняет, что за объект перед ним и как его умножать. Массив — один сплошной кусок памяти, где числа одного типа лежат подряд. Цикл по нему идёт внутри numpy, в скомпилированном коде, без этих выяснений, и `a * 1.8 + 32` на миллионе чисел работает в десятки раз быстрее цикла.

> **Правило.** Искать операцию над целым массивом, а не писать цикл по элементам. Это главное правило урока, и дальше почти каждый раздел — о том, какие для этого есть операции.

2 Тип элементов

Раз числа лежат подряд, они одного типа. Тип хранится в поле `dtype`:

```
print(a.dtype, (a * 1.8).dtype)
print(np.array([1, 2.5]))
print(np.array([1, 2.5]).dtype)
```

```
int64 float64
[1.  2.5]
float64
```

`int64` — целые, 64 бита на число; `float64` — дробные. Если в списке есть хоть одно дробное число, дробным становится весь массив, и целое `1` напечаталось как `1.`, то есть 1,0.

Ловушка. Массив не меняет тип, когда в него записывают. Дробное число, записанное в целый массив, молча теряет дробную часть:

```
b = np.array([10, 20, 30])
b[0] = 3.9
print(b)
```

```
[ 3 20 30]
```

Одна строка в списке делает строками все элементы:

```
s = np.array([1, 2, "3"])
print(s, s.dtype)
```

```
['1' '2' '3'] <U21
```

`<U21` — строки до 21 знака. Складываться как числа такой массив не будет. При чтении файлов это частая беда: одна кривая запись — и столбец чисел оказался столбцом строк.

Тип меняет `astype`. Он не трогает исходный массив, а возвращает новый:

```
print(s.astype(int) + 1)
x = np.array([2.7, -2.7, 0.5, 1.5])
print(x.astype(int))
print(np.round(x), np.floor(x))
```

```
[2 3 4]
[ 2 -2  0  1]
[ 3. -3.  0.  2.] [ 2. -3.  0.  1.]
```

`astype(int)` отбрасывает дробную часть, то есть округляет к нулю, а не вниз: $-2{,}7$ стало -2. Вниз округляет `np.floor`. `np.round` округляет к ближайшему, а половину — к чётному: 0,5 дало 0, 1,5 дало 2. Так округляет и сам питон, это не ошибка.

Деление тоже зависит от типа:

```
print(a / 2)
print(a // 2)
print(a % 2)
```

```
[ 1.5  2.5  2.  -0.5 -1.   0.   3. ]
[ 1  2  2 -1 -1  0  3]
[1 1 0 1 0 0 0]
```

`/` всегда даёт дробные числа. `//` — деление нацело с округлением вниз, поэтому `-1 // 2` равно -1, а не 0; `%` — остаток, и он не бывает отрицательным при положительном делителе.

3 Как создать массив

Из списка — `np.array`. Но чаще массив строят сразу:

```
print(np.arange(5))
print(np.arange(2, 11, 3))
print(np.linspace(0, 1, 5))
print(np.zeros(3), np.ones(4, dtype=int))
print(np.full(3, 7.5))
```

```
[0 1 2 3 4]
[2 5 8]
[0.   0.25 0.5  0.75 1.  ]
[0. 0. 0.] [1 1 1 1]
[7.5 7.5 7.5]
```

`np.arange(начало, конец, шаг)` устроен как `range`: конец в него не входит. `np.linspace(0, 1, 5)` — пять точек через равные промежутки, и оба конца входят. `zeros`, `ones` и `full` заполняют массив нулями, единицами и заданным числом; тип по умолчанию дробный, другой задают параметром `dtype`.

Ловушка. Дробный шаг у `arange` опасен: сколько выйдет точек, решает округление.

```
print(np.arange(1, 1.2, 0.1))
print(np.arange(1, 1.3, 0.1))
```

```
[1.  1.1]
[1.  1.1 1.2 1.3]
```

В первой строке конец 1,2 не вошёл, как и обещано, а во второй конец 1,3 вошёл. Число точек `arange` считает как $\lceil (1{,}3-1)/0{,}1 \rceil$, а в дробных числах это частное чуть больше трёх — и точек выходит четыре. Для дробных точек — `linspace`: у него число точек задано прямо, `np.linspace(1, 1.2, 3)`.

4 Номера и срезы

Элементы нумеруются с нуля, отрицательный номер считается с конца. Срез `a[i:j]` берёт номера от `i` до `j`, не включая `j`; пропущенный край — до начала или до конца:

```
print(a[0], a[3], a[-1])
print(a[1:4])
print(a[:3], a[5:])
print(a[::2], a[::-1])
```

```
3 -1 6
[ 5  4 -1]
[3 5 4] [0 6]
[ 3  4 -2  6] [ 6  0 -2 -1  4  5  3]
```

Третье число в срезе — шаг: `::2` — через один, `::-1` — задом наперёд. Пока всё как у списков. Новое — запись: в срез пишут разом, и число растягивается на все его места.

```
w = a.copy()
w[:5] = 0
print(w)
```

```
[0 0 0 0 0 0 6]
```

`copy` здесь не для красоты — зачем он, в разделе 6. Номера можно дать и списком: выйдет массив из выбранных элементов, в любом порядке и с повторами.

```
print(a[[0, 6, 6, 2]])
```

```
[3 6 6 4]
```

Пример (соседи). Насколько каждый день теплее предыдущего. Два среза, сдвинутых на один, ставят каждый день напротив вчерашнего:

```
print(a[1:])
print(a[:-1])
print(a[1:] - a[:-1])
```

```
[ 5  4 -1 -2  0  6]
[ 3  5  4 -1 -2  0]
[ 2 -1 -5 -1  2  6]
```

Семь дней дают шесть разностей. Для этого есть и готовая функция, `np.diff`, — она будет в разделе 13; но приём со сдвинутыми срезами работает для любой формулы от соседей, не только для разности.

5 Таблица: две оси

Теперь наблюдения в трёх городах — три строки по семь дней. Названия городов — отдельным массивом строк, в том же порядке:

```
T = np.array([[ 3,  5,  4, -1, -2,  0,  6],
              [-4, -6, -1, -8, -9, -5, -2],
              [12, 17, 11, 10, 13, 15, 16]])
city = np.array(["Москва", "Мурманск", "Сочи"])
print(T.shape, T.ndim, T.size)
```

```
(3, 7) 2 21
```

Форма $(3, 7)$: по оси 0 три города, по оси 1 семь дней. `ndim` — число осей, `size` — всего элементов. Номер по каждой оси пишут через запятую:

```
print(T[1, 3])
print(T[2])
print(T[:, 0])
print(T[:, -2:])
```

```
-8
[12 17 11 10 13 15 16]
[ 3 -4 12]
[[ 0  6]
 [-5 -2]
 [15 16]]
```

`T[1, 3]` — Мурманск, четверг. Один номер — строка целиком. Двоеточие значит «все»: `T[:, 0]` — понедельник во всех городах, то есть столбец, а `T[:, -2:]` — выходные, срез по обеим осям сразу.

Ловушка. `T[1][3]` тоже даёт Мурманск в четверг: сначала строка, потом элемент в ней. Но столбец так не взять:

```
print(T[:][0])
print(T[:, 0])
```

```
[ 3  5  4 -1 -2  0  6]
[ 3 -4 12]
```

`T[:]` — вся таблица, и `[0]` после неё — её первая строка. Столбец — только через запятую.

Форму можно поменять, не трогая числа: `reshape` раскладывает те же элементы по строкам заново, в том же порядке.

```
r = np.arange(12)
print(r.reshape(3, 4))
print(r.reshape(-1, 6))
print(T.T.shape)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
(7, 3)
```

-1 в `reshape` значит «сколько получится»: 12 чисел по 6 в строке — две строки. `T.T` — транспонирование: строки становятся столбцами. Обратно в одну строку раскладывает `ravel()`. Сложить таблицы друг на друга — `np.stack`, приставить строки или столбцы — `np.concatenate` с нужной осью.

6 Срез — не копия

Ловушка. Срез не копирует данные, а смотрит в те же. Запись в срез меняет исходный массив:

```
row = T[0]
row[0] = 100
print(T[0])
```

```
[100   5   4  -1  -2   0   6]
```

Москва в понедельник теперь $+100$, хотя `T` никто не трогал.

Вернём на место и сделаем как надо — копией:

```
T[0, 0] = 3
safe = T[0].copy()
safe[0] = 100
print(T[0])
```

```
[ 3  5  4 -1 -2  0  6]
```

Как устроено. Массив — это кусок памяти и описание, как по нему ходить: форма и шаг по каждой оси. Срез — новое описание над тем же куском: другое начало, другая форма, другой шаг. Поэтому срез берётся мгновенно при любом размере и поэтому же запись в него видна в исходном. `reshape` и `.T` — тоже новые описания тех же данных. А выбор по списку номеров и по маске (следующий раздел) собирает элементы в новый массив — это уже копия:

```
print(np.shares_memory(T, T[:, 1:]))
print(np.shares_memory(T, T[[0, 2]]))
```

```
True
False
```

Правило. Функция не портит массив, который ей передали. Если надо менять на месте — сначала `x = x.copy()`. А лучше писать без изменений на месте: `np.sort(x)` вместо `x.sort()`, `x = x + 1` вместо `x += 1`.

7 Маски

Сравнение массива с числом тоже идёт поэлементно, и выходит массив из `True` и `False` — маска:

```
warm = T > 0
print(warm)
print(warm.dtype)
```

```
[[ True  True  True False False False  True]
 [False False False False False False False]
 [ True  True  True  True  True  True  True]]
bool
```

Маской выбирают: `T[warm]` — все тёплые дни подряд, одной строкой. А при счёте `True` — это 1, `False` — 0, так что сумма маски — сколько раз условие выполнено, а среднее — в какой доле случаев:

```
print(T[warm])
print(warm.sum(), warm.mean())
```

```
[ 3  5  4  6 12 17 11 10 13 15 16]
11 0.5238095238095238
```

Условия соединяют знаками `&` — «и», `|` — «или», `~` — «не»; каждое условие в скобках:

```
print(T[(T > 0) & (T < 10)])
print(((T < -5) | (T > 14)).sum())
print(T[~warm].size)
```

```
[3 5 4 6]
6
10
```

Ловушка. Слова `and` и `or` здесь не работают:

```
print(T[(T > 0) and (T < 10)])
```

```
ValueError: The truth value of an array with more
than one element is ambiguous. Use a.any() or
a.all()
```

`and` спрашивает у массива одно «да» или «нет», а у него их двадцать одно. Скобки тоже обязательны: `&` сильнее сравнения, и `T > 0 & T < 10` читается как `T > (0 & T) < 10`.

Одно «да» или «нет» на весь массив дают `any` — есть ли хоть один `True` — и `all` — все ли:

```
print((T < 0).any(), (T < 0).all())
```

```
True False
```

По маске можно и записывать. Морозы — в ноль, на копии:

```
D = T.copy()
D[D < 0] = 0
print(D[:2])
```

```
[[3 5 4 0 0 0 6]
 [0 0 0 0 0 0 0]]
```

Без записи то же делает `np.where(условие, a, b)` — поэлементно «если, то, иначе»; `a` и `b` могут быть числами или массивами той же формы. А номера, где маска верна, — `np.flatnonzero`:

```
print(np.where(T < 0, 0, T)[0])
print(np.flatnonzero(a < 0))
```

```
[3 5 4 0 0 0 6]
[3 4]
```

Пример (теплее среднего). Сколько дней в Москве было теплее, чем в среднем за неделю, и какие это дни по номерам:

```
hot = a > a.mean()
print(hot.sum(), np.flatnonzero(hot))
```

```
4 [0 1 2 6]
```

8 Счёт вдоль оси

`T.sum()` складывает всё. С параметром `axis` счёт идёт вдоль одной оси, и эта ось пропадает:

```
print(T.sum())
print(T.sum(axis=1))
print(T.mean(axis=0))
```

```
74
[ 15 -35  94]
[3.66666667 5.33333333 4.66666667 0.33333333
 0.66666667 3.33333333 6.66666667]
```

Вдоль оси 1, по дням, — сумма за неделю в каждом городе: три числа. Вдоль оси 0, по городам, — средняя по трём городам за каждый день: семь чисел.

> **Правило.** `axis` называет ту ось, которой не станет. У `T` формы $(3, 7)$ счёт по оси 0 даёт форму $(7,)$, по оси 1 — $(3,)$.

Так же считают `max`, `min`, `std`, `any`, `all`, `argmax`. Последний возвращает не само наибольшее, а его номер:

```
print(T.max(axis=1))
print(T.argmax(axis=1))
print(city[T.mean(axis=1).argmax()])
```

```
[ 6 -1 17]
[6 2 1]
Сочи
```

Вторая строка — в какой день недели в каждом городе было теплее всего. Номер можно сразу подставить в другой массив: последняя строка нашла самый тёплый город.

Пример (без мороза). В каких городах за неделю ни разу не было мороза. Маска по дням, потом `all` вдоль дней, потом выбор городов маской по оси 0:

```
no_frost = (T >= 0).all(axis=1)
print(no_frost, city[no_frost])
```

```
[False False  True] ['Сочи']
```

Осей бывает больше. Пусть есть и прошлая неделя; `np.stack` кладёт две таблицы друг на друга, и выходит форма $(2, 3, 7)$ — неделя, город, день:

```
W = np.stack([T - 2, T])
print(W.shape)
print(W.mean(axis=(0, 2)))
print(W.max(axis=0).shape)
```

```
(2, 3, 7)
[ 1.14285714 -6.         12.42857143]
(3, 7)
```

`axis=(0, 2)` сворачивает две оси сразу — недели и дни, — и остаётся по числу на город. Правило то же: какие оси

названы, тех и не станет. Ещё один параметр, `keepdims=True`, оставляет свёрнутую ось на месте, длиной 1:

```
print(T.mean(axis=1).shape)
print(T.mean(axis=1, keepdims=True).shape)
```

```
(3,)
(3, 1)
```

Зачем это нужно — в разделе 10.

9 Пропуски

В Сочи в среду термометр не работал. Пустого числа в numpy нет, и пропуск пишут особым дробным значением `np.nan`, «не число». В целый массив его не записать:

```
T[2, 2] = np.nan
```

```
ValueError: cannot convert float NaN to integer
```

Поэтому сначала копия в дробных:

```
S = T.astype(float)
S[2, 2] = np.nan
print(S[2])
print(S[2].mean(), S.sum(axis=1))
```

```
[12. 17. nan 10. 13. 15. 16.]
nan [ 15. -35.  nan]
```

Пропуск заражает любой счёт: среднее и сумма с ним — тоже `nan`. Для данных с пропусками у функций есть варианты с

приставкой `nan`, и они пропуски обходят. Маску пропусков даёт `np.isnan`:

```
print(np.nanmean(S[2]), np.nanmax(S, axis=1))
print(np.isnan(S).sum(), np.isnan(S).any(axis=1))
```

```
13.833333333333334 [ 6. -1. 17.]
1 [False False  True]
```

Ловушка. Сравнение с `nan` всегда ложно, даже с самим собой. Поэтому пропуски нельзя искать через `== np.nan`:

```
print(np.nan == np.nan)
print((S == np.nan).sum(), np.isnan(S).sum())
```

```
False
0 1
```

Заполнить пропуск — тот же `np.where`, только с маской пропусков; например, средним по остальным дням того же города:

```
s = S[2]
filled = np.where(np.isnan(s), np.nanmean(s), s)
print(filled.round(1))
```

```
[12.  17.  13.8 10.  13.  15.  16. ]
```

Если вдоль оси одни пропуски, считать не из чего: выходит `nan` и предупреждение. Это не ошибка, программа идёт дальше, но ответ надо проверить:

```
print(np.nanmean(np.array([np.nan, np.nan])))
```

```
nan
RuntimeWarning: Mean of empty slice
```

10 Распространение

Массив плюс число — число прибавляется к каждому элементу. Это частный случай общего правила: массивы разной формы растягиваются друг под друга. По-английски это broadcasting.

> **Правило.** Формы сравнивают с конца, ось за осью. На каждой оси длины совпадают или одна из них равна 1 — тогда она растягивается до другой. Если у одного массива оси кончились, недостающие считаются длиной 1.

Термометры в городах врут, каждый на своё число градусов: у Москвы на градус больше правды, у Мурманска на полградуса меньше, у Сочи точно. Поправка — три числа, по одному на строку:

```
corr = np.array([-1, 0.5, 0])
print(T + corr)
```

```
ValueError: operands could not be broadcast together
with shapes (3,7) (3,)
```

С конца: 7 против 3 — не совпали, и ни одна не 1. Нужен не ряд из трёх, а столбец. `None` на месте номера вставляет новую ось длиной 1, и форма $(3,)$ становится $(3, 1)$:

```
print(corr[:, None].shape)
print(T + corr[:, None])
```

```
(3, 1)
[[ 2.   4.   3.  -2.  -3.  -1.   5. ]
 [-3.5 -5.5 -0.5 -7.5 -8.5 -4.5 -1.5]
 [12.  17.  11.  10.  13.  15.  16. ]]
```

Теперь с конца: 7 против 1 — растягивается, 3 против 3 — совпали. Отсюда и `keepdims`: средняя по городу формы $(3, 1)$

сразу вычитается из каждой строки — выходит отклонение от средней своего города.

```
dev = T - T.mean(axis=1, keepdims=True)
print(dev.round(1))
```

```
[[ 0.9  2.9  1.9 -3.1 -4.1 -2.1  3.9]
 [ 1.  -1.   4.  -3.  -4.   0.   3. ]
 [-1.4  3.6 -2.4 -3.4 -0.4  1.6  2.6]]
```

Ряд формы $(7,)$ растягивается на все строки и без `None`: формы $(3, 7)$ и $(7,)$ совпадают с конца. Вычесть из каждого дня среднюю по городам за этот день:

```
print((T - T.mean(axis=0)).round(1))
```

```
[[ -0.7  -0.3  -0.7  -1.3  -2.7  -3.3  -0.7]
 [ -7.7 -11.3  -5.7  -8.3  -9.7  -8.3  -8.7]
 [  8.3  11.7   6.3   9.7  12.3  11.7   9.3]]
```

Столбец и ряд вместе растягиваются в таблицу всех пар — без единого цикла:

```
k = np.arange(1, 6)
print(k[:, None] * k)
print(np.abs(k[:, None] - k))
```

```
[[ 1  2  3  4  5]
 [ 2  4  6  8 10]
 [ 3  6  9 12 15]
 [ 4  8 12 16 20]
 [ 5 10 15 20 25]]
[[0 1 2 3 4]
 [1 0 1 2 3]
 [2 1 0 1 2]
 [3 2 1 0 1]
 [4 3 2 1 0]]
```

Первая таблица — умножения, вторая — расстояния между всеми парами чисел. Таблица «каждый с каждым» — частый

приём: так считают разности, расстояния и сравнения всех пар.

Как устроено. Растяжение ничего не копирует: по оси длины 1 numpy идёт с шагом 0 и берёт тот же элемент снова. Память уходит только на результат. Но у таблицы пар он из $n \cdot m$ чисел, и при $n = m = 10^5$ это 10^{10} чисел — столько в память не влезет.

11 Сортировка

`np.sort` — копия по возрастанию. `np.argsort` — не сами числа, а номера: где стоял самый маленький, где следующий за ним, и так далее.

```
print(np.sort(a))
print(np.sort(a)[::-1])
print(np.argsort(a))
```

```
[-2 -1  0  3  4  5  6]
[ 6  5  4  3  0 -1 -2]
[4 3 5 0 2 1 6]
```

Номера нужны, чтобы переставить так же другой массив. Города по средней температуре, от холодного к тёплому, и вместе с ними их строки таблицы:

```
order = np.argsort(T.mean(axis=1))
print(city[order])
print(T[order][:, :3])
```

```
['Мурманск' 'Москва' 'Сочи']
[[-4 -6 -1]
 [ 3  5  4]
 [12 17 11]]
```

Три самых тёплых дня Москвы, от тёплого — три последних номера `argsort`, задом наперёд:

```
print(np.argsort(a)[-3:][::-1])
```

```
[6 1 2]
```

Ловушка. Порядок равных `argsort` не обещает, если не попросить `kind="stable"`: тогда равные остаются в исходном порядке, меньший номер раньше.

```
v = np.array([2, 1, 2, 1, 2])
print(np.argsort(v, kind="stable"))
print(np.argsort(v, kind="stable")[::-1])
print(np.argsort(-v, kind="stable"))
```

```
[1 3 0 2 4]
[4 2 0 3 1]
[0 2 4 1 3]
```

По убыванию — сортировать `-v`. Перевернуть сортировку по возрастанию нельзя: перевернутся и равные, и больший номер окажется раньше.

Сортировка лежит и под порядковыми статистиками: медиана — середина отсортированного, процентиль — точка, ниже которой заданная доля значений.

```
print(np.median(a), np.percentile(a, 90))
print(np.sort(T, axis=1)[:, -1])
```

```
3.0 5.4
[ 6 -1 17]
```

Сортировать можно и вдоль оси: каждая строка отдельно. Последний столбец отсортированных строк — наибольшее в каждом городе, то же, что `T.max(axis=1)`.

12 Счёт по группам

Группы — когда у каждого элемента есть метка и считать надо по меткам. Пусть у каждого дня Москвы записана погода и осадки в миллиметрах:

```
sky = np.array(["ясно", "дождь", "ясно", "снег",
                "снег", "ясно", "дождь"])
rain = np.array([0, 4, 0, 2, 3, 0, 6])
u, cnt = np.unique(sky, return_counts=True)
print(u, cnt)
```

```
['дождь' 'снег' 'ясно'] [2 2 3]
```

`np.unique` — различные значения по возрастанию, с `return_counts` — ещё и сколько раз встречается каждое. Сколько осадков выпало в дни одного вида — маска: `rain[sky == "снег"].sum()`. Но маска на каждую группу — это снова цикл, по группам. Всё сразу делает пара `unique` и `bincount`:

```
u, g = np.unique(sky, return_inverse=True)
print(g)
print(np.bincount(g))
print(np.bincount(g, weights=rain))
```

```
[2 0 2 1 1 2 0]
[2 2 3]
[10.  5.  0.]
```

`return_inverse` даёт номер группы у каждого дня: 0 — дождь, 1 — снег, 2 — ясно, по порядку `u`. `np.bincount(g)` считает, сколько раз встретился каждый номер, а с `weights` — складывает веса по номерам. Все группы за один проход.

Как устроено. `bincount` заводит счётчики — столько, каков наибольший номер плюс один, — и один раз проходит по данным, прибавляя к счётчику своего номера. Поэтому номера — небольшие целые с нуля: при номере в миллиард

счётчики займут гигабайты. Если групп больше, чем встретилось, `minlength` добавляет пустые, со счётом 0. Метки любого вида сначала переводят в номера через `unique`.

13 Накопления и окна

`np.cumsum` — накопленные суммы: на месте k сумма первых $k+1$ чисел. `np.diff` — разности соседних, обратная к ней операция:

```
print(np.cumsum(rain))
print(np.diff(a))
print(np.diff(np.cumsum(rain)))
```

```
[ 0  4  4  6  9  9 15]
[ 2 -1 -5 -1  2  6]
[4 0 2 3 0 6]
```

Сумма осадков за любые три дня подряд — разность двух накопленных сумм. Чтобы формула работала с первого дня, впереди ставят ноль: тогда `c[i]` — сумма первых `i` дней, а сумма с дня `i` по день `i + 2` — это `c[i + 3] - c[i]`.

```
c = np.concatenate([[0], np.cumsum(rain)])
print(c)
print(c[3:] - c[:-3])
```

```
[ 0  0  4  4  6  9  9 15]
[4 6 5 5 9]
```

Окна можно и увидеть. `sliding_window_view` показывает все окна строками и ничего при этом не копирует — это вид, как срез:

```
swv = np.lib.stride_tricks.sliding_window_view
win = swv(a, 3)
print(win)
print(win.mean(axis=1).round(2))
```

```
[[ 3  5  4]
 [ 5  4 -1]
 [ 4 -1 -2]
 [-1 -2  0]
 [-2  0  6]]
[ 4.    2.67  0.33 -1.    1.33]
```

Дальше — любой счёт вдоль оси 1: здесь скользящее среднее за три дня. Окон длины 3 у семи чисел пять, и у первых двух дней своего окна нет. Если нужен результат той же длины, что данные, начало дополняют, например, `nan`; в pandas неполные окна устроены удобнее, см. урок о таблицах.

14 Векторы и матрицы

Знак `@` — произведение матриц, матрицы на вектор и скалярное произведение векторов. Средняя температура дня с весами городов 0,5, 0,3 и 0,2 — вектор весов на таблицу:

```
wt = np.array([0.5, 0.3, 0.2])
print(wt @ T)
print(T @ np.ones(7) / 7)
```

```
[ 2.7  4.1  3.9 -0.9 -1.1  1.5  5.6]
[ 2.14285714 -5.         13.42857143]
```

Форма $(3,)$ на $(3,7)$ даёт $(7,)$: по взвешенному среднему на день. $(3,7)$ на $(7,)$ даёт $(3,)$: таблица на вектор из единиц делённых на 7 — средняя каждого города.

Как устроено. `@` — это суммы произведений, то есть тот же счёт по оси после распространения:

```
s1 = wt @ T
s2 = (wt[:, None] * T).sum(axis=0)
print(np.allclose(s1, s2))
```

```
True
```

Но `@` зовёт библиотеку линейной алгебры, написанную под процессор, и на больших матрицах он быстрее в разы. `np.allclose` — «равны с точностью до округлений»: дробные числа через `==` не сравнивают.

Длина вектора — `np.linalg.norm`, скалярное произведение — тот же `@`, а из них косинус угла между векторами:

```
p = np.array([3.0, 4.0])
q = np.array([1.0, 0.0])
norm = np.linalg.norm
print(norm(p), p @ q)
print(p @ q / (norm(p) * norm(q)))
```

```
5.0 3.0
0.6
```

У таблицы `np.linalg.norm(X, axis=1)` — длины всех строк сразу. Систему уравнений $Ax = b$ решает `np.linalg.solve`:

```
A = np.array([[2.0, 1.0], [1.0, 3.0]])
b = np.array([3.0, 5.0])
x = np.linalg.solve(A, b)
print(x, A @ x)
```

```
[0.8 1.4] [3. 5.]
```

15 Случайные числа

Случайные числа даёт генератор с зерном. С тем же зерном выходят те же числа — опыт можно повторить, и ответ у всех одинаковый:

```
rng = np.random.default_rng(42)
print(rng.integers(1, 7, size=10))
print(rng.normal(0, 1, size=3).round(3))
print(rng.choice(city, size=4))
print(rng.permutation(5))
```

```
[1 5 4 3 3 6 1 5 2 1]
[-1.302  0.128 -0.316]
['Мурманск' 'Москва' 'Сочи' 'Мурманск']
[4 0 2 3 1]
```

`integers(1, 7)` — кубик: от 1 до 6, конец не входит. `normal(0, 1)` — нормальное распределение со средним 0 и разбросом 1, `choice` — случайный выбор из массива, `permutation` — случайная перестановка.

Пример (два кубика). Как часто сумма двух кубиков равна 7. Сто тысяч бросков — таблица $(100\,000, 2)$, сумма по оси 1, маска, среднее:

```
d = rng.integers(1, 7, size=(100_000, 2))
print((d.sum(axis=1) == 7).mean())
```

```
0.16784
```

Точный ответ — $6/36 \approx 0{,}1667$. Подчёркивание в `100_000` питон пропускает: это только для глаз.

16 Файлы

Массивы хранят в файлах numpy: `.npy` — один массив, `.npz` — несколько, каждый под своим именем:

```
np.save("t.npy", T)
print(np.load("t.npy").shape)
np.savez("week.npz", T=T, city=city)
f = np.load("week.npz")
print(f.files)
print(f["city"])
```

```
(3, 7)
['T', 'city']
['Москва' 'Мурманск' 'Сочи']
```

`np.load` у `.npz` даёт что-то вроде словаря: имена в `f.files`, массив — по имени в квадратных скобках. Текстовые таблицы numpy читает тоже (`np.loadtxt`), но для них есть pandas, и с ним удобнее.

17 Итог: как обойтись без цикла

Почти каждый раздел был заменой какого-то цикла:

- поэлементная формула — вместо цикла по элементам;
- маска и `np.where` — вместо `if` внутри цикла;
- счёт вдоль оси — вместо цикла по строкам или столбцам;
- распространение и `None` — вместо двойного цикла по парам;
- `argsort` — вместо поиска лучшего;
- `unique` и `bincount` — вместо словаря со счётчиками;
- `cumsum`, `diff` и окна — вместо цикла по соседям и окнам.

Ловушка. `np.vectorize` и `np.apply_along_axis` цикл не убирают, а прячут: внутри они вызывают функцию питона на каждом элементе или строке, и быстрее от них не становится.

Дальше — урок «Таблицы в pandas». Там те же массивы, но у столбцов есть имена, у строк — метки, в одной таблице уживаются числа, строки и даты, а группы и окна считаются одной командой.